FPGA experiment report

Task: FPGA 4

Student 1 name: Mahmood Stitia ID: 214032682

Student 2: name: Saleh Khalil ID: 213310485

Part 1 - Status update

Fill in the following table according to your progress at the time of the submission. Each row should refer to one of the modules together with its simulation.

The status column should be filled with one of the following:

- Done - If both are written and simulation passed.
- Partially done - If you started writing the module and simulation but it's not working properly.
- Not started - If you didn't get to that part of the task.

When the status is partially done, please use the notes/comments column to explain in a few more words what is the status of the module and what is the status of the simulation.

Fill here:

Module name/Step	status	notes/comments
Control	Done	TB Passed
Debouncer	Done	
Stopwatch	Done	
Elaborated Design	Done	
Synthesis	Done	
Setting up constraints	Done	
Implementing the design	Done	
FPGA programming and debugging	Done	

Issues:

In this section please describe the current issues you are facing.

- Per each module and simulation, if you receive any errors at the time of submission, please describe which errors and what they mean.

Fill here:

- Control:
- Debouncer:
- Stopwatch:
- Elaborated Design:
- Synthesis:
- Setting up constraints:
- Implementing the design:
- FPGA programming and debugging:

Part 2 - Simulations and answers

In this section, please add the images for each simulation as described in the report. Remember to use annotations and explain in your own words what is shown in each simulation.

In some of the tasks in the report, there are also questions that you should answer here.

Make sure to include images of all simulations running at the time of submission, even if they are not correct.

Fill here:

- **Control:**

  2c. Test-bench. The test-bench exhaustively drives the FSM from each of the three reachable states (`IDLE`, `COUNTING`, `PAUSED`) and applies every possible combination of `{reset,trig,split}` (`cii` = $0 \ldots 7$).

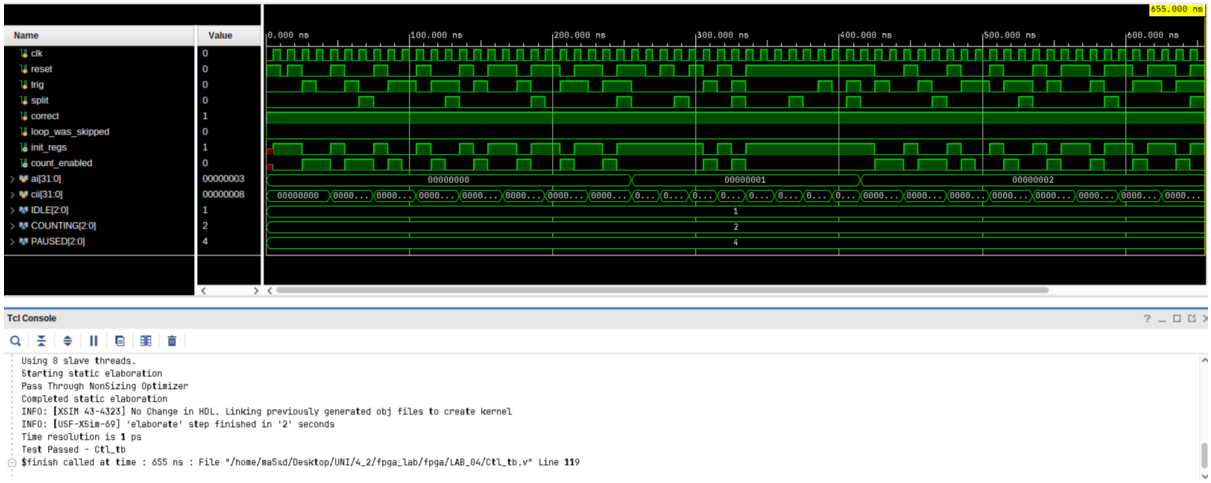


Figure 1: Annotated waveform of `Ctl_tb`. The outer counter `ai` sweeps the three starting states (IDLE $=0$, COUNTING $=1$, PAUSED $=2$) and the inner counter `cii` sweeps the 8 input combinations of `{reset,trig,split}`. The `correct` signal stays high for the whole simulation, and the TCL console at the bottom shows `Test Passed - Ctl_tb` and Corrent was high all the time

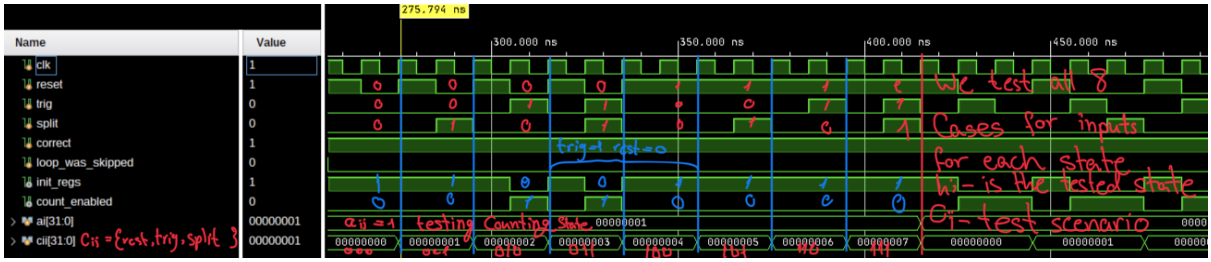


Figure 2: Zoomed view inside the `ai = 1` (COUNTING) block. After the short preamble (one `reset` pulse, then one `trig` pulse to leave IDLE), the FSM is in COUNTING and `count_enabled` is high. Each subsequent `cii` value tests one of the COUNTING transitions in Figure 1: a `trig` pulse drops `count_enabled` (move to PAUSED) and a later `reset` forces `init_regs` high (return to IDLE).

2d.ii Mealy vs. Moore. The assignment draws the FSM in Mealy form (each arrow shows *both* inputs and outputs). However, all transitions *leaving the same state* carry the *same output*:

- every arrow leaving `IDLE` outputs `init_regs,count_enabled = 10`,
- every arrow leaving `COUNTING` outputs `01`,
- every arrow leaving `PAUSED` outputs `00`.

The output therefore depends only on the current state, so this FSM is in fact a **Moore** FSM. This is exactly how the output is written in `Ctl.v`:

$$\texttt{init\_regs} = (\texttt{state} == \texttt{IDLE}), \qquad \texttt{count\_enabled} = (\texttt{state} == \texttt{COUNTING}).$$

- **Debouncer:**

  3c. High vs. low `COUNTER_BITS` – the tradeoff.

  The hysteresis counter must climb from 0 to $2^{\texttt{COUNTER\_BITS}-1}$ for the MSB to flip and emit a single-cycle pulse. With a 100 MHz clock the debounce window is therefore

  $$T_{\text{debounce}} \approx \frac{2^{\texttt{COUNTER\_BITS}-1}}{f_{\text{clk}}}.$$

  For the value we use in the top module ($\texttt{COUNTER\_BITS} = 20$), $T_{\text{debounce}} \approx 2^{19}/10^8\,\text{s} \approx 5.24\,\text{ms}$, comfortably above the typical 1–2 ms contact bounce of a tactile button.

  The `COUNTER_BITS` parameter determines the duration of the debounce interval. Increasing its value enlarges the counter, which improves immunity to noise and contact bouncing but also introduces longer latency before the system responds to a button press. Conversely, reducing this value shortens response time but increases the likelihood of false triggering caused by unstable or noisy input signals.

  This tradeoff is directly analogous to the low-pass-filter-based debouncing approach from earlier experiments. A lower cutoff frequency averages the input over a longer time, suppressing bouncing at the cost of increased delay, while a higher cutoff frequency allows faster response with reduced noise suppression. Similarly, a larger counter value requires longer input stability, whereas a smaller counter value reacts faster but is more sensitive to noise.

- **Stopwatch:**

  4a. Block diagram and connectivity.

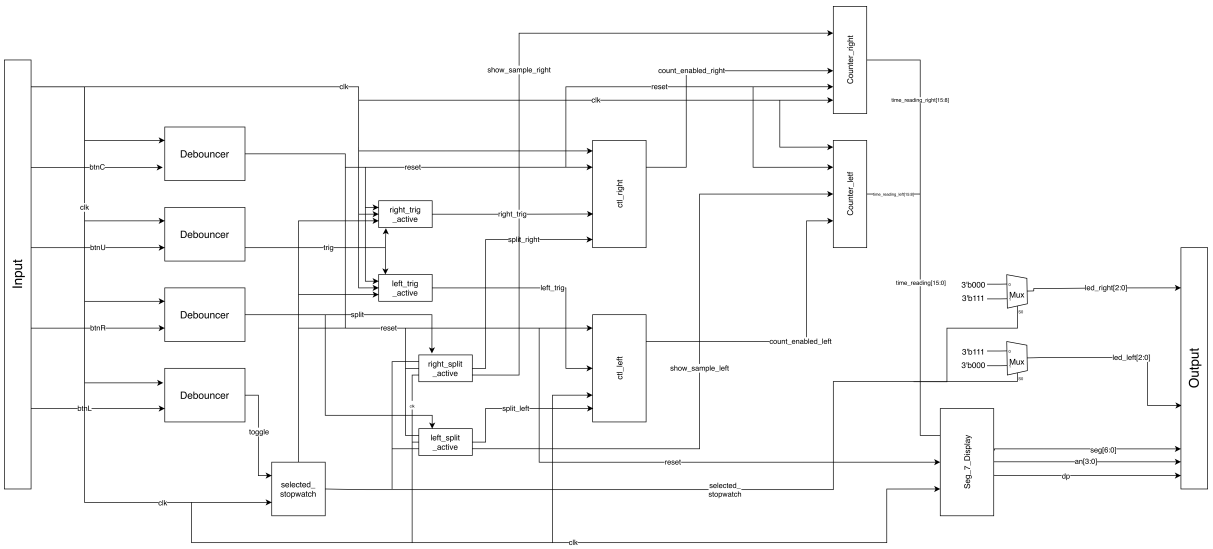


Figure 3: block diagram for the Stopwatch module

Input Signals:

clk	Clock input
btnC	Center Button input
btnU	Upper Button input
btnR	Right Button input
btnL	Left Button input

Output Signals:

seg[6:0]	7-segment display control
an[3:0]	Anode control for multiplexed display
dp	Decimal point control
led_left[2:0]	Left set of LEDs
led_right[2:0]	Right set of LEDs

- **Elaborated Design:**

  5b. Elaborated-design schematic.

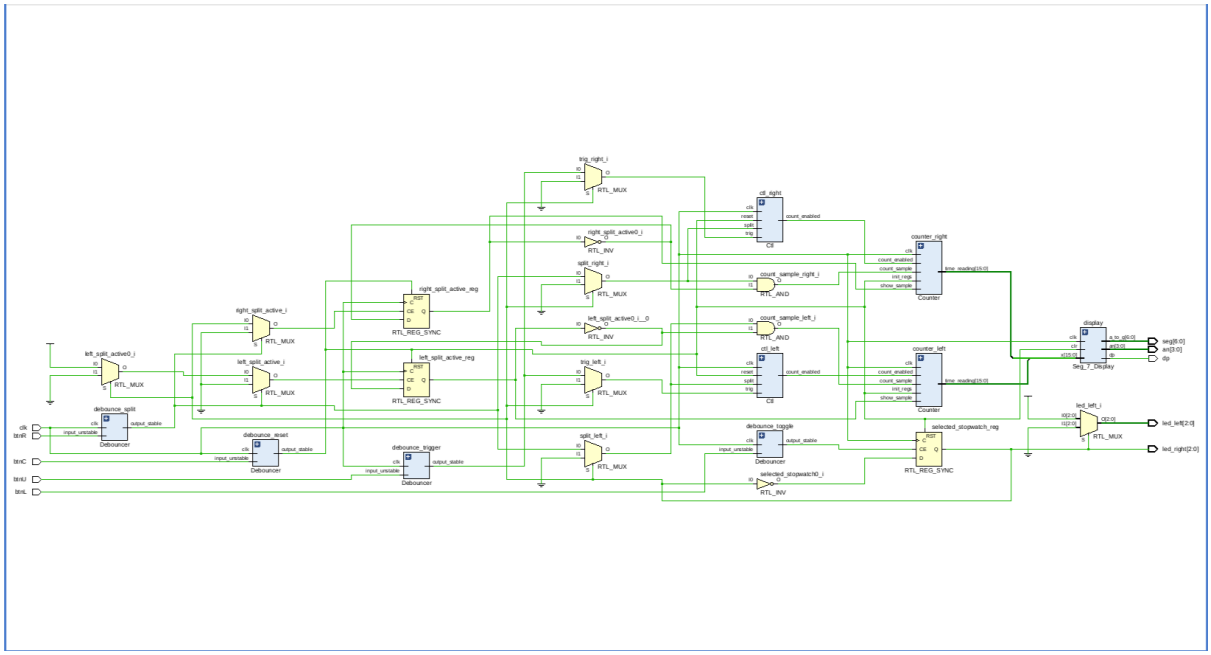


Figure 4: RTL-elaborated schematic of `Stopwatch`. All four `Debouncer` instances, the two `Ctl` blocks, the two `Counter` blocks and the shared `Seg_7_Display` appear as hierarchical boxes. The orange `RTL_MUX` gates implement the input routing (trig / split go only to the selected side), `RTL_AND` gates produce the `count_sample` pulses (`split & ~split_active`), and the synchronous `RTL_REG_SYNC` elements are the `selected_stopwatch`, `left_split_active` and `right_split_active` registers. All top-level ports (`btnC`, `btnU`, `btnR`, `btnL`, `clk` on the left and `seg[6:0]`, `an[3:0]`, `dp`, `led_left[2:0]`, `led_right[2:0]` on the right) are present.

- **Synthesis:**

  6c.i Synthesized schematic.

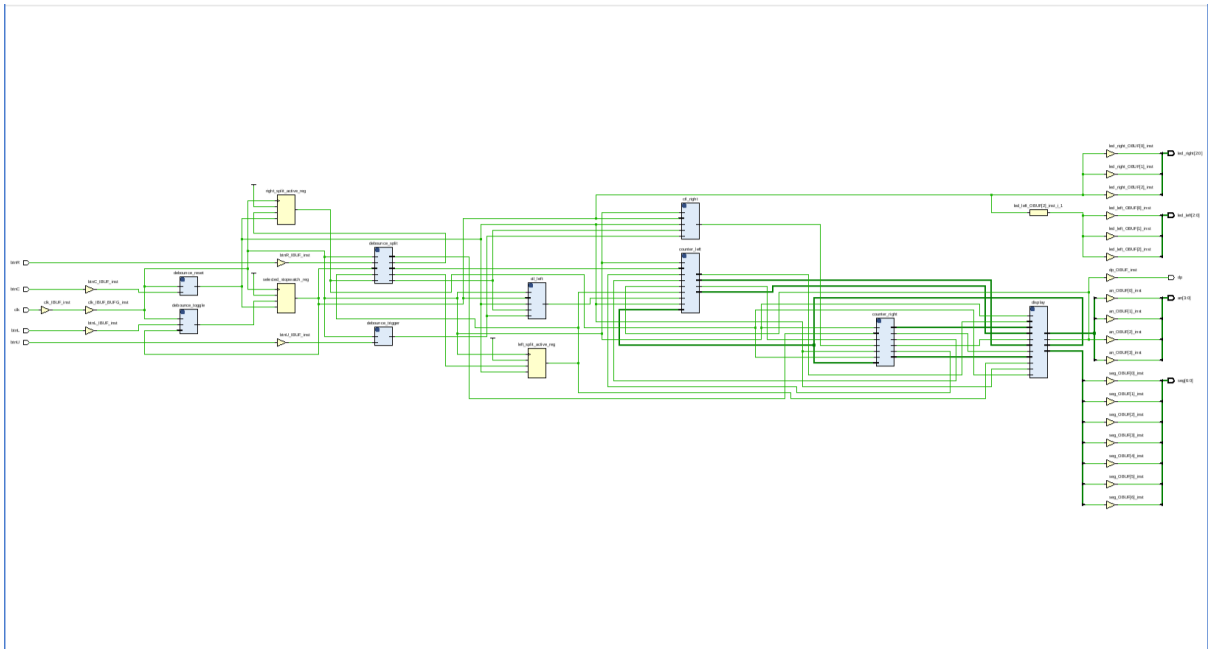


Figure 5: Post-synthesis schematic of `Stopwatch`. The hierarchical boxes are still recognisable, but every external signal now passes through an `IBUF` / `OBUF` I/O buffer, the clock goes through a `BUFG` global buffer, and the combinational logic inside each block has been mapped onto LUTs and FDRE/FDSE flip-flops. The `led_left` / `led_right` OBUFs on the right show that synthesis recognised that the three LEDs of each side share the same drive signal (`selected_stopwatch` or its inverse).

6c.ii Differences between elaborated and synthesized. In the elaborated design, we see the logical/behavioral structure as written in Verilog—high-level modules, generic RTL primitives (RTL_MUX, RTL_REG, RTL_AND), and abstract signal buses. After synthesis, Vivado maps the design onto actual FPGA primitives: LUTs (Look-Up Tables), FDREs/FDSEs (flip-flops with reset/set and enable), IBUFs/OBUFs (I/O buffers), and a BUFG (global clock buffer). The synthesizer also performs optimizations such as merging logic, removing unused paths, and flattening hierarchies where beneficial.

Implementing the design:

8c.i Timing summary.

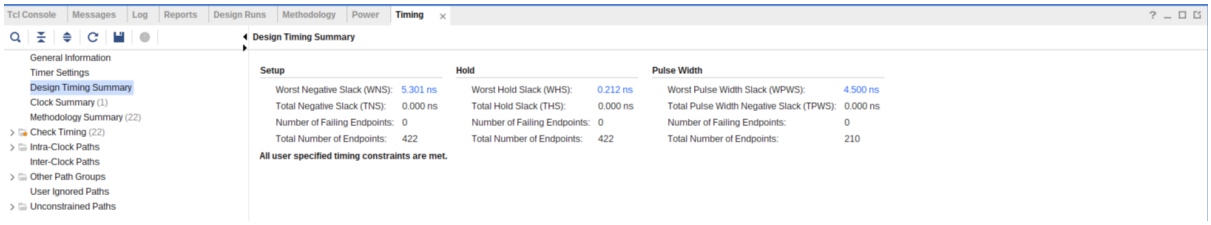


Figure 6: Design Timing Summary after implementation. All user-specified timing constraints are met: the Setup, Hold and Pulse-Width slacks are all positive, with zero failing endpoints in every category.

The timing summary reports:

- **Setup:** Worst Negative Slack (WNS) $= 5.301$ ns – positive $\Rightarrow$ constraint met.
- **Hold:** Worst Hold Slack (WHS) $= 0.212$ ns – positive $\Rightarrow$ constraint met.
- **Pulse Width:** Worst Pulse-Width Slack (WPWS) $= 4.500$ ns – positive $\Rightarrow$ constraint met.
- Number of Failing Endpoints: 0 in every category (out of 422 setup/hold endpoints and 210 pulse-width endpoints).

8c.ii Critical setup path length.

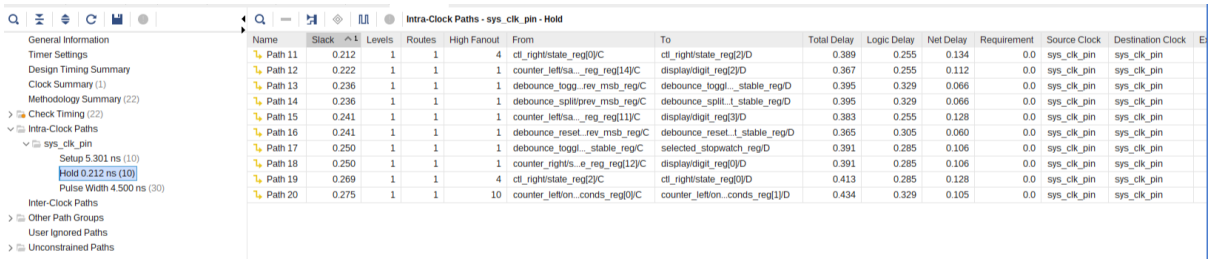
Working from the WNS first, the setup slack is the gap between the clock period and the longest data path:

$$\begin{aligned} \text{slack}_{\text{setup}} &= T_{\text{clk}} - T_{\text{critical}} - T_{\text{uncertainty}} \\ 5.301\,\text{ns} &\approx 10\,\text{ns} - T_{\text{critical}} \\ T_{\text{critical}} &\approx 4.699\,\text{ns}. \end{aligned}$$

Vivado's Intra-Clock Paths report confirms this directly:

Figure 7: Intra-Clock Paths – Setup. The critical setup path (Path 1) runs from `counter_left/clk_cnt_reg[19]/C` to `counter_left/on...seconds_reg[0]/D`, with a total datapath delay of 4.743 ns (1.212 ns logic + 3.531 ns net), 4 logic levels and a slack of 5.301 ns against the 10 ns requirement.

The reported total path length is therefore $\boxed{T_{\text{critical}} = 4.743\,\text{ns}}$. The very small difference from the 4.699 ns derived from the slack is the clock uncertainty/jitter term that Vivado subtracts separately. For completeness, the worst hold path is shown below:



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 11	0.212	1	1	4	ctl_right/state_reg[0]/C	ctl_right/state_reg[2]/D	0.389	0.255	0.134	0.0	sys_clk_pin	sys_clk_pin
Path 12	0.222	1	1	1	counter_left/sa..._reg_reg[14]/C	display/digit_reg[2]/D	0.367	0.255	0.112	0.0	sys_clk_pin	sys_clk_pin
Path 13	0.236	1	1	1	debounce_togg...rev_msb_reg/C	debounce_toggl..._stable_reg/D	0.395	0.329	0.066	0.0	sys_clk_pin	sys_clk_pin
Path 14	0.236	1	1	1	debounce_split/prev_msb_reg/C	debounce_split...t_stable_reg/D	0.395	0.329	0.066	0.0	sys_clk_pin	sys_clk_pin
Path 15	0.241	1	1	1	counter_left/sa..._reg_reg[11]/C	display/digit_reg[3]/D	0.383	0.255	0.128	0.0	sys_clk_pin	sys_clk_pin
Path 16	0.241	1	1	1	debounce_reset...rev_msb_reg/C	debounce_reset...t_stable_reg/D	0.365	0.305	0.060	0.0	sys_clk_pin	sys_clk_pin
Path 17	0.250	1	1	1	debounce_toggl..._stable_reg/C	selected_stopwatch_reg/D	0.391	0.285	0.106	0.0	sys_clk_pin	sys_clk_pin
Path 18	0.250	1	1	1	counter_right/s...e_reg_reg[12]/C	display/digit_reg[0]/D	0.391	0.285	0.106	0.0	sys_clk_pin	sys_clk_pin
Path 19	0.269	1	1	4	ctl_right/state_reg[2]/C	ctl_right/state_reg[0]/D	0.413	0.285	0.128	0.0	sys_clk_pin	sys_clk_pin
Path 20	0.275	1	1	10	counter_left/on...conds_reg[0]/C	counter_left/on...conds_reg[1]/D	0.434	0.329	0.105	0.0	sys_clk_pin	sys_clk_pin

Figure 8: Intra-Clock Paths – Hold. The worst hold path (Path 11) runs inside `ctl_right` between two adjacent state-register bits, with a total delay of 0.389 ns and a positive slack of 0.212 ns, confirming that no flip-flop violates its hold time.

FPGA programming and debugging:

9a-b. On hardware the two stopwatches behave as specified:

- `btnL` toggles the selection between the left and right stopwatches; the `led_left` / `led_right` bars indicate which side is active.
- `btnU` (trig) starts and pauses *only* the currently selected stopwatch; the other side keeps running unaffected.
- `btnR` (split) freezes / unfreezes the displayed value of the currently selected side without stopping the underlying counter.
- `btnC` (reset) zeroes both stopwatches and brings the selection back to the left side.

9c. Post-implementation project summary. After verifying the design on hardware, the debug core was removed and the design was re-implemented to obtain the final post-implementation reports:

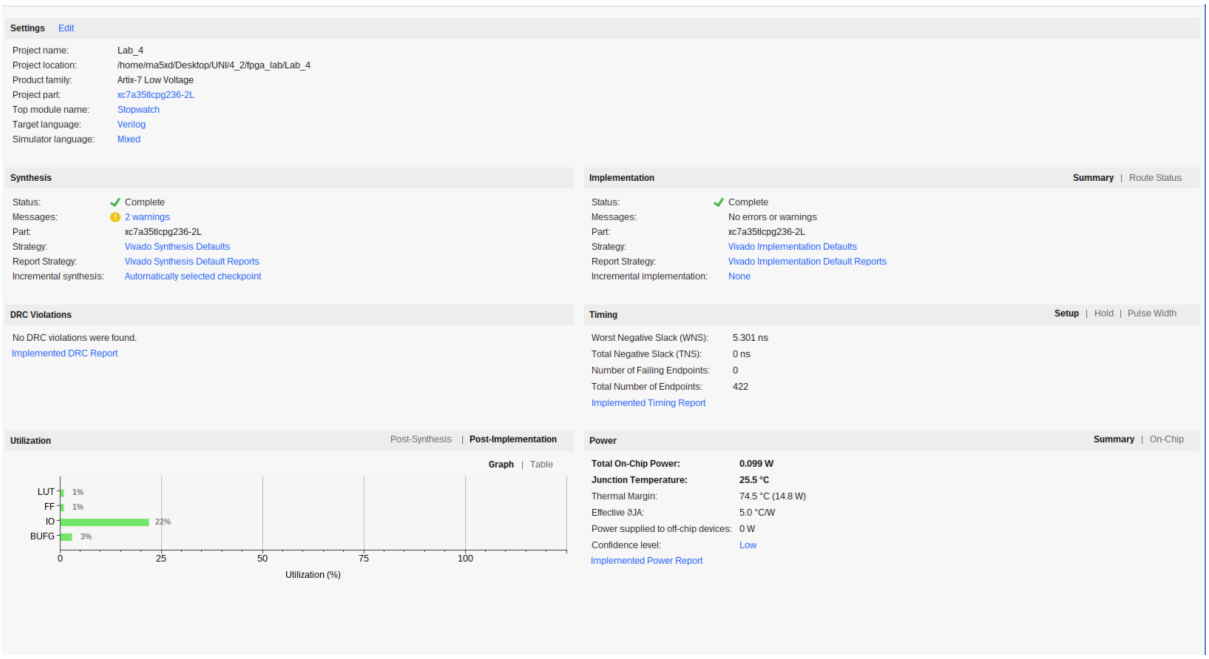


Figure 9: Project Summary after implementation

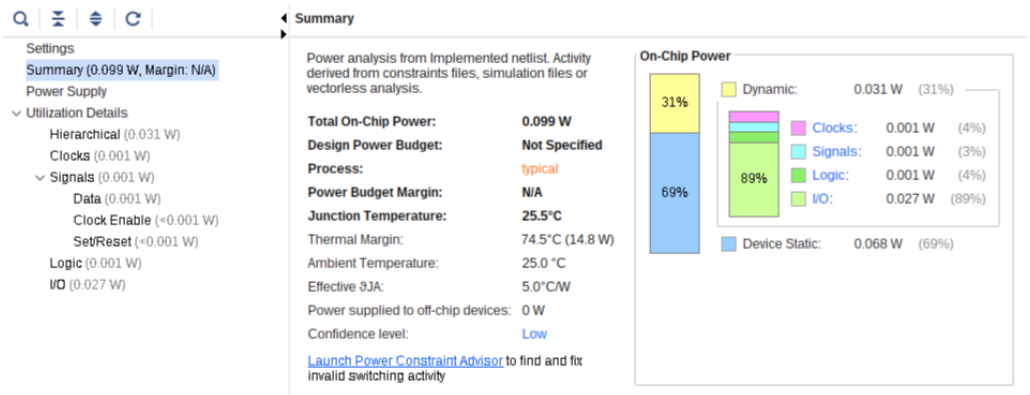


Figure 10: Power report. Total on-chip power is $0.099\,\mathrm{W}$, of which $0.031\,\mathrm{W}$ (31%) is dynamic (mostly I/O at $0.027\,\mathrm{W}$) and $0.068\,\mathrm{W}$ (69%) is device static. Junction temperature is $25.5°\mathrm{C}$ with a thermal margin of $74.5°\mathrm{C}$ ($14.8\,\mathrm{W}$).

Appendix – Verilog Source Code

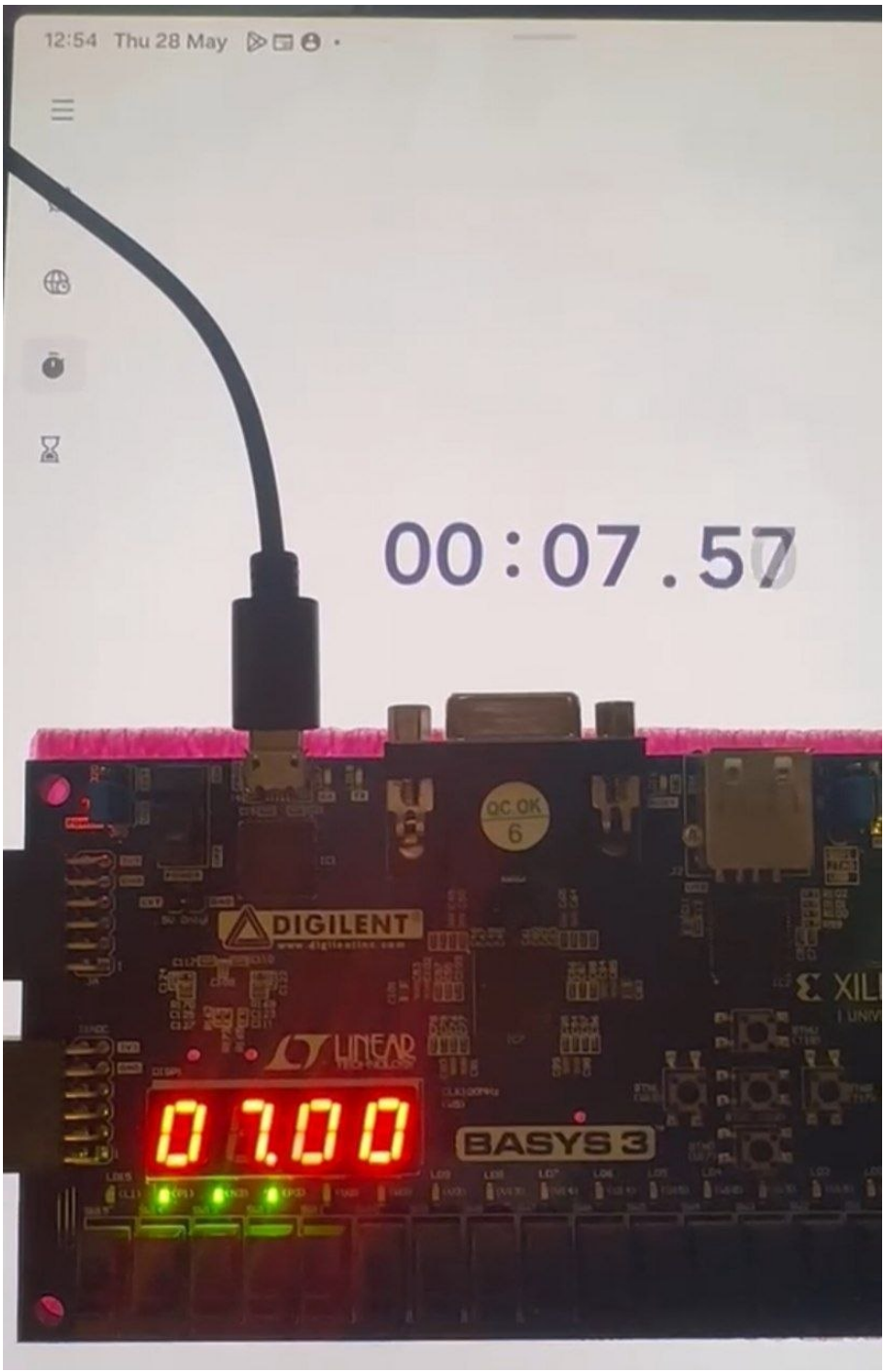


Figure 11: Stopwatch at 07.00 seconds, with the left side selected and split active. The left two digits show the frozen value of the left stopwatch (07), while the right two digits show the live value of the right stopwatch (00).

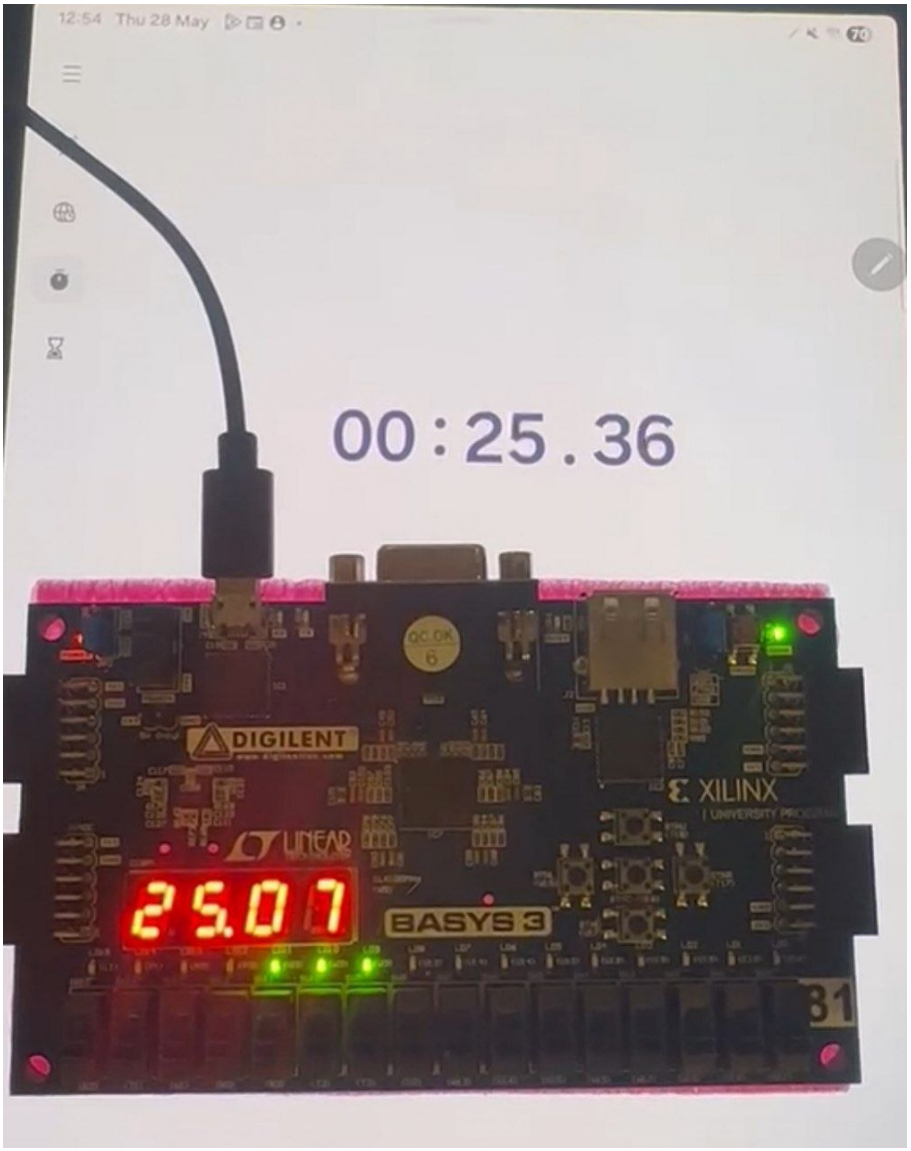


Figure 12: Stopwatch at 25.07 seconds, with the right side selected as the LED indicator shows. The left two digits show the value of the left stopwatch (25), while the right two digits show the value of the right stopwatch (07).

Ctl.v

```
`timescale 1ns/10ps
module Ctl(clk, reset, trig, split, init_regs, count_enabled);

   input  clk, reset, trig, split;
   output init_regs, count_enabled;

   localparam SIZE = 3;
   localparam IDLE = 3'b001, COUNTING = 3'b010, PAUSED = 3'b100;
   reg [SIZE-1:0] state;

   always @(posedge clk) begin
      if (reset)
         state <= IDLE;
      else begin
         case (state)
            IDLE     : state <= (trig)? COUNTING : IDLE;
            COUNTING : state <= (trig)? PAUSED   : COUNTING;
            PAUSED   : begin
               casez({reset, trig, split})
                  3'b000 : state <= PAUSED;
                  3'b001 : state <= IDLE;
                  3'b01? : state <= COUNTING;
                  3'b1?? : state <= IDLE;
                  default: state <= PAUSED;
               endcase
            end
            default  : state <= IDLE;
         endcase
      end
   end

   assign init_regs     = (state == IDLE);
   assign count_enabled = (state == COUNTING);

endmodule
```

Ctl_tb.v

```
module Ctl_tb();
    reg clk, reset, trig, split, correct, loop_was_skipped;
    wire init_regs, count_enabled;
    integer ai, cii;
    localparam IDLE = 3'b001, COUNTING = 3'b010, PAUSED = 3'b100;
    Ctl uut(.clk(clk), .reset(reset), .trig(trig), .split(split),
            .init_regs(init_regs), .count_enabled(count_enabled));
    initial begin
        correct = 1; loop_was_skipped = 0;
        ai = 0; cii = 0;
        clk = 0; reset = 1; trig = 0; split = 0;
        #10 reset = 0;
        correct = correct & init_regs & ~count_enabled;
        @(posedge clk);
        loop_was_skipped = 1;
        for (ai = 0; ai < 3; ai = ai + 1) begin
            loop_was_skipped = 0;
            for (cii = 0; cii < 8; cii = cii + 1) begin
                reset = 1; #10 reset = 0;
                if (ai != IDLE)   begin trig = 1; #10 trig = 0; end
                if (ai == PAUSED) begin trig = 1; #10 trig = 0; end
                {reset, trig, split} = cii;
                #10
                {reset, trig, split} = 0;
                case (ai)
                    IDLE: casez(cii)
                        3'b00? : correct = correct & init_regs & ~
                            count_enabled & (uut.state == IDLE);
                        3'b1?? : correct = correct & init_regs & ~
                            count_enabled & (uut.state == IDLE);
                        3'b01? : correct = correct & ~init_regs &
                            count_enabled & (uut.state == COUNTING);
                        default: correct = 0;
                    endcase
                    COUNTING: casez(cii)
                        3'b1?? : correct = correct & init_regs & ~
                            count_enabled & (uut.state == IDLE);
                        3'b00? : correct = correct & ~init_regs &
                            count_enabled & (uut.state == COUNTING);
                        3'b01? : correct = correct & ~init_regs & ~
                            count_enabled & (uut.state == PAUSED);
                        default: correct = 0;
                    endcase
                    PAUSED: casez(cii)
                        3'b1?? : correct = correct & init_regs & ~
                            count_enabled & (uut.state == IDLE);
                        3'b01? : correct = correct & ~init_regs &
                            count_enabled & (uut.state == COUNTING);
                        3'b001 : correct = correct & init_regs & ~
                            count_enabled & (uut.state == IDLE);
                        3'b000 : correct = correct & ~init_regs & ~
                            count_enabled & (uut.state == PAUSED);
                        default: correct = 0;
                    endcase
                endcase
            end
        end
        if (correct) $display("Test Passed - %m");
        else         $display("Test Failed - %m");
        $finish;
    end
```

```
    always #5 clk = ~clk;
endmodule
```

Debouncer.v

```
`timescale 1ns/10ps
module Debouncer(clk, input_unstable, output_stable);

   input  clk, input_unstable;
   output reg output_stable;

   parameter COUNTER_BITS = 7;
   reg prev_msb;
   reg [COUNTER_BITS-1:0] counter;

   always @(posedge clk) begin
      if (input_unstable == 1)
         counter <= (counter < {COUNTER_BITS{1'b1}}) ? counter + 1 : counter;
      else
         counter <= (counter > {COUNTER_BITS{1'b0}}) ? counter - 1 : counter;

      prev_msb      <= counter[COUNTER_BITS-1];
      output_stable <= (~prev_msb) & counter[COUNTER_BITS-1];
   end

endmodule
```

Stopwatch.v

```
1  `timescale 1ns/10ps
2  module Stopwatch(clk, btnC, btnU, btnR, btnL, seg, an, dp, led_left,
       led_right);
3
4      input              clk, btnC, btnU, btnR, btnL;
5      output  wire [6:0] seg;
6      output  wire [3:0] an;
7      output  wire       dp;
8      output  wire [2:0] led_left;
9      output  wire [2:0] led_right;
10
11     wire [15:0] time_reading;
12     // button signals
13     wire trig, split, reset, toggle;
14     // counter signals
15     wire trig_right, split_right, init_regs_right, count_enabled_right,
           count_sample_right, show_sample_right;
16     wire trig_left,  split_left,  init_regs_left,  count_enabled_left,
           count_sample_left, show_sample_left;
17     // selected stopwatch
18     localparam LEFT = 1'b0, RIGHT = 1'b1;
19     reg selected_stopwatch; //left -> 0; right -> 1
20
21     // FILL HERE INSTANTIATIONS
22     wire [15:0] time_reading_left, time_reading_right;
23     // reg right_split_active, left_split_active;
24
25     ////////////////////////////
26     // Buttons Stabilization //
27     ////////////////////////////
28     Debouncer #(.COUNTER_BITS(20)) debounce_reset(
29         .clk(clk),
30         .input_unstable(btnC),
31         .output_stable(reset)
32     );
33
34     Debouncer #(.COUNTER_BITS(20)) debounce_trigger(
35         .clk(clk),
36         .input_unstable(btnU),
37         .output_stable(trig)
38     );
39
40     Debouncer #(.COUNTER_BITS(20)) debounce_split(
41         .clk(clk),
42         .input_unstable(btnR),
43         .output_stable(split)
44     );
45
46     Debouncer #(.COUNTER_BITS(20)) debounce_toggle(
47         .clk(clk),
48         .input_unstable(btnL),
49         .output_stable(toggle)
50     );
51
52     ////////////////////////////
53     //        Controls       //
54     ////////////////////////////
55     Ctl ctl_right(
56         .clk(clk),
57         .reset(reset),
58         .trig(trig_right),
```

```
59          .split(split_right),
60          .init_regs(init_regs_right),
61          .count_enabled(count_enabled_right)
62      );
63      Ctl ctl_left(
64          .clk(clk),
65          .reset(reset),
66          .trig(trig_left),
67          .split(split_left),
68          .init_regs(init_regs_left),
69          .count_enabled(count_enabled_left)
70      );
71
72      ////////////////////////////
73      //   7-Segment Display    //
74      ////////////////////////////
75      Seg_7_Display display(
76          .x(time_reading),
77          .clk(clk),
78          .clr(reset),
79          .a_to_g(seg),
80          .an(an),
81          .dp(dp)
82      );
83
84      ////////////////////////////
85      //        Counters        //
86      ////////////////////////////
87      Counter counter_right(
88          .clk(clk),
89          .init_regs(reset | init_regs_right),
90          .count_enabled(count_enabled_right),
91          .count_sample(count_sample_right),
92          .show_sample(show_sample_right),
93          .time_reading(time_reading_right)
94      );
95      Counter counter_left(
96          .clk(clk),
97          .init_regs(reset | init_regs_left),
98          .count_enabled(count_enabled_left),
99          .count_sample(count_sample_left),
100         .show_sample(show_sample_left),
101         .time_reading(time_reading_left)
102     );
103
104     ////////////////////////////
105     //    Stopwatch Logic     //
106     ////////////////////////////
107
108     always @(posedge clk) begin // TOGGLE SELECTED STOPWATCH
109         if(reset) begin
110             selected_stopwatch <= LEFT;
111         end
112         else begin
113             if(toggle) selected_stopwatch <= ~selected_stopwatch;
114         end
115     end
116
117
118     // left stopwatch signals
119     assign led_left  = (selected_stopwatch == LEFT)? 3'b111 : 3'b000; // left
        stopwatch is selected
120     assign trig_left   = (selected_stopwatch == LEFT)? trig  : 1'b0;
```

```
121     assign split_left  = (selected_stopwatch == LEFT)? split : 1'b0;
122     assign show_sample_left  = 1'b0;
123     assign count_sample_left = 1'b0;
124 
125     // right stopwatch signals
126     assign led_right = (selected_stopwatch == RIGHT)? 3'b111 : 3'b000; //
            right stopwatch is selected
127     assign trig_right  = (selected_stopwatch == RIGHT)? trig  : 1'b0;
128     assign split_right = (selected_stopwatch == RIGHT)? split : 1'b0;
129     assign show_sample_right  = 1'b0;
130     assign count_sample_right = 1'b0;
131 
132     assign time_reading = {time_reading_left[15:8], time_reading_right
            [15:8]};
133 
134 endmodule
```